ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ. ПАКЕТЫ.

ЛАБОРАТОРНАЯ РАБОТА №3

Решение всех задач оформить в одном проекте, но в разных классах. В главном методе main показать работу всех классов и методов с дружественным интерфейсом. Все исходные данные вводятся с клавиатуры, организовать проверку на ввод. В каждом классе должны присутствовать свойства, конструкторы и метод toString().

Необходимо решить задания согласно вашему варианту. Задачи из заданий 1-6 оцениваются по 0,8 баллов, задачи из заданий 7-8 – 0,6 балла. Максимально за лабораторную работу можно получить 10 баллов (8 баллов за решение задач + 2 балла за оформление отчета).

Распределение вариантов

№варианта	№1	№2	№3	№4	№5	№6	№7	№8
1	6,12	1	5	5	1	3	1,2,3	5
2	4,10	5	1	2	1	5	1,2,3	4
3	7,8	2	5	3	4	2	1,2,3	4
4	5,7	3	4	1	8	2	1,2,3	2
5	10	5	3,5	3	9	5	1,2,3	4
6	1,5	1	4	1	8	2	1,2,3	2
7	2,11	4	5	3	2	2	1,2,3	1
8	4,7	2	1	2	1	1	1,2,3	3
9	9	3	5	4	3,6	2	1,2,3	4
10	3	4	2	4	5,7	4	1,2,3	4

Задание 1. Инкапсуляция: private, геттеры и сеттеры, выбрасывание исключений через throw, класс RuntimeException и его наследники.

1	**Дом над землей.** Измените сущность Дом из задачи **1.4.3**. Гарантируйте, что у дома всегда будет положительное количество этажей. В случае попытки указать отрицательное количество этажей должна выбрасываться соответствующая ошибка. Продемонстрируйте работоспособность решения на примерах.
2	**Время от начала суток**. Измените сущность Время из задачи **1.5.4**. Гарантируйте, что значение, хранящееся в поле данного класса всегда будет обозначать время прошедшее с начала суток. В случае если при создании объекта вводится слишком большое число, все, что превышает 24 часа отбрасывается. Ранее написанные методы должны быть изменены так, чтобы они отталкивались от предположения, что хранящееся число – верное. Продемонстрируйте работоспособность решения на примерах.

3	**Квадрат и Сторона Квадрата.** Создайте сущность Квадрат. Квадрат описывается следующими характеристиками: • Имеет Точку (из задачи **1.4.1**) отмечающую левый верхний угол • Имеет длину стороны. • Создается путем указания точки левого верхнего угла и размера стороны • Создается путем указания координаты **x**и **y** левого верхнего угла и размера стороны • Может быть приведен к строке вида “Квадрат в точке T со стороной N”, где N – длина стороны, а T – результат приведения к строке верхнего левого угла • Может возвращать новую Ломануюю (из задачи 1.5.7), точки которой соответствуют точкам углов текущего квадрата. Необходимо выполнить следующие задачи: • Создайте Квадрат в точке {5;3} со стороной 23 • Присвойте в ссылку типа Ломаная результат вызова метода получения Ломаной у ранее созданного квадрата • Выведите на экран общую длину полученной Ломаной • Сдвиньте последнюю Точку Ломаной в позицию {15,25} • Снова выведите на экран длину Ломаной Измените сущность Квадрат. Гарантируйте, что Квадрату невозможно задать отрицательное значение длины стороны. При попытке задать сторону равной нулю или менее, выбрасывается ошибка с соответствующим текстом. Контроль должен осуществляться как во время создания объекта, так и позднее, в случае если будет необходимо менять размеры Квадрата. Необходимо предоставить возможность изменения длины стороны Квадрата и возможность узнать размер стороны. Продемонстрируйте работоспособность решения на примерах.
4	**Дроби.** Измените сущность Дробь из задачи **1.5.5**. Реализуйте следующие требования: • Дробь не может быть изменена после создания • Необходимо корректно обрабатывать отрицательные значения. Учтите, что знаменатель не может быть отрицательным. Продемонстрируйте работоспособность решения на примерах.
5	**Перезарядка Пистолета.** Измените сущность Пистолет из задачи **1.5.1**. Модификация предполагает внесение следующих дополнительных требований: • Имеет максимальное количество патронов. Максимальное количество устанавливается во время создания пистолета и не может быть изменено позднее. У пистолета можно узнать, какое максимальное количество он вмещает. • Может быть перезаряжен. Для перезарядки необходимо передать пистолету число, которое будет означать количество заряжаемых патронов. Если передано отрицательное число, необходимо выбросить ошибку, объясняющую, что отрицательного числа патронов быть не может. Если передано слишком большое число патронов – необходимо лишние вернуть. • Может быть разряжен. Это приводит к обнулению патронов в пистолете и возврате нужного числа пользователю. • Можно узнать сколько сейчас заряжено патронов. • Можно узнать заряжен он или разряжен. Создайте пистолет вместимостью 7, зарядите три патрона, выстрелите из него пять раз, затем зарядите в него 8 патронов, выстрелите еще 2 раза, разрядите его, сделайте контрольный выстрел. Если все выполнено верно, то должно быть выведено: Бах! Бах! Бах! Клац! Клац! Бах! Бах! Клац!
6	**Отдельные линии.** Измените сущность Линия из задачи **1.5.3**. Необходимо, чтобы Линия соответствовала следующим требованиям: • Две любые линии не могут ссылаться на один и тот же объект точки. • У Линии можно изменить координаты начала или конца • У Линии можно запросить координаты начала или конца Продемонстрируйте работоспособность решения на примерах.
7	**Непустые Имена.** Измените сущность Имя из задачи **1.4.5**. Гарантируйте, что: • Как минимум один параметр будет иметь не **null** значение и не пустую строку. • Имя неизменяемо. Продемонстрируйте работоспособность решения на примерах.

8	**Человек меняется**. Измените сущность Человек из задачи **1.4.6.** Новые требования включают: • Рост человека всегда положительный и не более 500, его можно изменить и получить в любой момент. • Отца возможно задать только при создании и в дальнейшем он не изменяется, но его можно получить • Имя можно получить в любой момент в виде объекта типа Имя. Продемонстрируйте работоспособность решения на примерах.
9	**Диапазон оценок.** Измените сущность Студент из задачи **1.5.6**. Необходимо гарантировать, что добавлять Студенту можно только оценки в диапазоне от 2 до 5, при этом у Студента всегда можно узнать список оценок. Продемонстрируйте на примерах, что нет способа задать Студенту некорректную оценку.
10	**Дороги**. Измените сущности из задачи **1.3.3.** Гарантируйте, что между двумя городами может быть только одна прямая дорога (другой путь может быть проложен только транзитом через другие города). Города можно создавать с указанием заранее заданных путей, в любой момент времени можно добавить новую дорогу в любой город и удалить имеющуюся дорогу.
11	**Начальник отдела.** Измените сущности полученные в задаче **1.2.4.** Необходимо гарантировать, что начальник отдела гарантированно работает в том же отделе, в котором он начальник.
12	**Квадрат.**Создайте сущность Квадрат. Квадрат описывается следующими характеристиками: • Имеет Точку (из задачи **1.4.1**) отмечающую левый верхний угол • Имеет длину стороны. • Создается путем указания точки левого верхнего угла и размера стороны • Создается путем указания координаты **x**и **y** левого верхнего угла и размера стороны • Может быть приведен к строке вида “Квадрат в точке T со стороной N”, где N – длина стороны, а T – результат приведения к строке верхнего левого угла • Может возвращать новую Ломаную (из задачи **1.5.7**), точки которой соответствуют точкам углов текущего квадрата. Необходимо выполнить следующие задачи: 1. Создайте Квадрат в точке {5;3} со стороной 23 2. Присвойте в ссылку типа Ломаная результат вызова метода получения Ломаной у ранее созданного квадрата 3. Выведите на экран общую длину полученной Ломаной 4. Сдвиньте последнюю Точку Ломаной в позицию {15,25} 5. Снова выведите на экран длину Ломаной

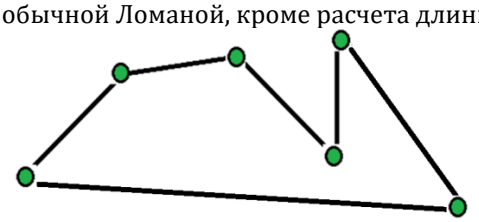
Задание 2. Разработка структур данных.

1	**Неизменяемый массив.** Необходимо разработать сущность НеизменяемыйСписокЗначений, представляющий собой обертку над массивом целых чисел, который сохранит функциональные возможности массивов, но добавит некоторые дополнительные возможности. Состояние сущности описывают следующие сведения: • Имеет массив целых чисел. Именно он используется для хранения значений. Инициализация сущности может быть выполнена следующим образом: • С указанием значений в виде массива целых чисел. • С указанием перечня чисел как независимых значений (через запятую) • С указанием другого Списка, в этом случае копируются все значения указанного списка. Поведение сущности описывают следующие действия: • Получение значения из позицииN. Позиция должна попадать в диапазон от 0 до N-1, где N–текущее количество значений, иначе выкинуть ошибку. • Замена значения в позиции N на новое значение. Позиция должна попадать в диапазон от 0 до N-1, где N–текущее количество значений, иначе выкинуть ошибку. • Может быть приведен к строке. Строка должна представлять собой полный перечень всех хранимых чисел, причем первый символ строки это ”[“, а последний “]”. • Можно проверить пуст Список или нет. Список пуст если его размер 0. • Можно узнать текущий размер. • Все хранящиеся значения можно запросить в виде стандартного массива. Продемонстрируйте работоспособность решения на примерах.
2	**Секреты.** Секреты никому нельзя рассказывать. Никому кроме одного самого близкого друга! Необходимо описать сущность Секрет, которая будет позволять хранить и передавать некоторый текст только одному другому хранителю. Состояние сущности описывают следующие сведения: • Текст секрета, в виде строки. • Имя хранителя секрета, в виде строки Инициализация сущности может быть выполнена следующим образом: • С указанием имени хранителя секрета и текста секрета, это будет означать создание нового секрета. • С указанием другого Секрета и именем хранителя секрета. В этом случае мы считаем, что секрет был рассказан другому человеку. При передаче секрета должно произойти следующее: во-первых, на консоль выводится текст вида “Имя сказал что Секрет”, где Имя и Секрет представляют собой имя того кто рассказывает секрет и текст секрета соответственно. Во-вторых, текст секрета у нового хранителя должен представлять собой копию текста предыдущего хранителя, но с добавлением X случайных символов в X случайных мест, где X — это число в диапазоне от 0 до N, а Nв свою очередь это 10% от размера исходного текста. Поведение сущности описывают следующие действия: • Может быть приведен к строке, строковое преставление вида“Имя: это секрет!”, где Имя – это конкретное имя хранителя секрета, а остальное простой текст. • Можно узнать каким по очереди был данных хранитель секрета. • Можно узнать сколько еще человек узнали секрет после текущего хранителя • Можно получить имя N-го человек узнавшего секрет, причем N будет положительным для случая, когда мы хотим узнать имя следующего узнавшего секрет, и отрицательным если предыдущего. • Можно узнать разницу в количестве символов текста секрета с N-ым человеком Необходимо учесть следующие требования к инкапсуляции: • Единственным способом получить текст секрета может быт его вывод на экран при инициализации объекта. • Секрет может быть передан только одному другому человеку и не должно быть способа рассказать один и тот же секрет нескольким людям. Продемонстрируйте работоспособность решения на примерах.

3	**Желтые страницы**. Необходимо разработать сущность ТелефонныйСправочник, который будет использоваться для создания справочника со списком телефонов и имен, с целью поиска телефона по имени. Будем считать достаточным хранение пар телефон-строка, например “89003337788 - Вася”. Состояние сущности описывается набором пар “телефон – имя”, причем и телефон и имя являются строками. В целях упрощения задачи будем считать, что одному имени всегда соответствует один телефон, и один телефон соответствует одному имени. Инициализация сущности выполняется без аргументов. Поведение сущности описывают следующие действия: • Добавление новой пары “телефон – имя” (как два отдельных аргумента). Конкретное местоположение добавленной пары не важно. Если добавляемое имя уже есть в списке, то имеющийся телефон заменяется на новый, и затем старый телефон возвращается из метода. • Удаление значения. Для удаления необходимо указать имя удаляемого контакта, после чего удаляется вся пара. • Получение значений. Для получения необходимо указать имя контакта, тогда возвращается его телефон. • Может быть приведен к строке. Строка должна представлять собой полный перечень всех хранимых пар. • Можно проверить есть ли конкретный телефон или конкретное имя в списке. • Можно узнать текущее количество контактов. • Можно запросить в виде массива: o Все пары o Все телефоны o Все имена • Можно получить массив всех имен, указав часть названия (начиная с первого символа). Продемонстрируйте работоспособность решения на примерах.
4	**Бинарное дерево.** Необходимо реализовать структуру данных, представляющую собой простейшее бинарное дерево. Для формирования данного дерева будет использоваться сущность Узел. Состояние сущности описывается следующими характеристиками: o Значение, целое число o Родитель, Узел o Левый потомок, Узел o Правый потомок, Узел. Инициализация сущности выполняется без аргументов. Поведение сущности описывают следующие действия: • Добавление значения. При добавлении значения возможно две ситуации o Для данного узла Значение не было задано ранее. В этом случае оно задается и работа завершается o Для данного узла Значение ранее было задано. В этом случае определяется, больше ли новое значение или меньше-равно чем текущее. Если больше – то значение добавляется в правый потомок, иначе в левый. Если нужный потомок отсутствует, то сначала он должен быть создан. • Удаление значения. В качестве аргумента передается значение, которое нужно удалить. Если такое значение есть, в самом узле или любом его потомке, то удаляется узел с этим значением, но все его потомки должны остаться прикрепленными к структуре. • Проверка на существование значения. В качестве аргумента передается значение, и возвращается информация о том есть ли такое значение в самом узле или любом его потомке. • Приведение к строке. Результатом приведения узла к строке является перечень всех значений хранимых в данном узле и всех его потомках, которое должно быть получено левосторонним обходом в глубину. После реализации сущности, создайте её экземпляр и добавить в него числа: 3 5 4 7 1 2. Выведите на экран текстовое представление узла.

5	**Маршрут.** Данная задача предполагает разработку новой сущности на основе той, что была получена в задаче **1.4.8** (Город). Сущность Город может быть доработана по своему усмотрению для более удобного использования. Основная идея задачи в разработке такой сущности, которая будет представлять собой маршрут между двумя городами. Данный маршрут в любой момент времени можно получить как массив для дальнейшего использования. Сущность Маршрут имеет следующие характеристики: • Имеет Город начала и Город конца пути. • Инициализация Маршрута может быть выполнена только если указана точка начала и конца пути. Если указано nullзначение - то ошибка. • Точку начала и конца можно изменить в любой момент времени, но они всегда должны существовать. • Может вернуть массив Городов, представляющий собой маршрут из начала в конец. Массив содержит все Города (в порядке очереди) через которые надо пройти что бы попасть из Города начала в Город конца, причем и начало и конец также содержатся в этом массиве. Алгоритм формирования пути в данном случае не существенен, можно выбрать вариант со случайным путем, путем проходящим через наименьшее число городов, или самым дешевым путем. Если путь найти невозможно – возвращается пустой массив. • Маршрут может быть приведен к строке, которая будет возвращать название всех городов маршрута в порядке очередности. • Создание объекта и изменение точек начала и конца выполняется за константное время O(1). Воспользуйтесь картой городов из задачи **1.3.3** и выведите маршрут из Города F в Город D

Задание 3. Наследование: extends, super, final, переопределение

1	**Запретная Дробь.** Измените сущность Дробь, полученную в задаче **2.1.4**. Гарантируйте, что невозможно создать такой подвид дроби, который позволял бы создавать Дроби с изменяемым состоянием.
2	**Замкнутая ломаная.** Создайте такой подвид сущности Ломанная, полученной в задаче **1.5.7**, которая будет представлять собой замкнутую ломанную линию, то есть последовательность точек, начинающихся и заканчивающихся на одной и той же точке (см. рисунок). Данная сущность во всех отношениях совпадает с обычной Ломаной, кроме расчета длины (по очевидным причинам).  Рис. Пример Замкнутой ломаной
3	**Двусторонняя дорога.** Создайте такой подвид сущности Город, полученной в задаче **2.1.10**, которая будет гарантировать, что при добавлении дороги из одного города в другой, одновременно будет добавляться и обратная дорога.

4	**Автомат**. Создайте такой подвид сущности Пистолет из задачи **2.1.5**, которая будет совпадать с ней во всех отношениях, кроме следующего: • Имеет скорострельность (целое число, неизменяемое) которое обозначает количество выстрелов в секунду, поддерживаемое данным автоматом. Скорострельность всегда положительное число. • При вызове Стрелять количество выстрелов соответствует скорострельности (например, при скорострельности 3 выводится три строки с текстом выстрела). • Умеет Стрелять N секунд, что приводит к количеству выстрелов равному N умноженное на скорострельность. • Инициализация может быть выполнены следующими способами: a) Без параметров. Скорострельность 30, вместимость 30. b) С указанием максимального числа патронов. Скорострельность будет равна половине обоймы c) С указанием максимального количества патронов в обойме и скорострельности.
5	**Трехмерная точка**. Создайте такой подвид сущности Точка из задачи **1.1.1**, которая будет иметь не две, а три координаты на плоскости: X,Y,Z.

Задание 4. Создание иерархий: abstract.

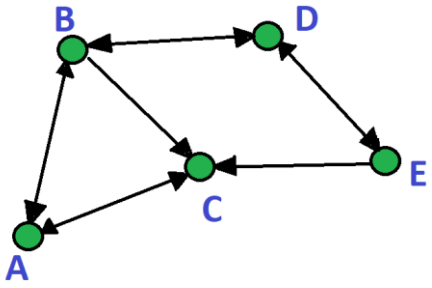
1	**Оружие.**
	Измените сущность Пистолет, полученную в задаче **2.1.5** таким образом, чтобы она наследовалась от класса **Weapon** описанного на рисунке:
	`abstract class Weapon{`
	`  private int ammo;`
	`  public Weapon(int ammo) {`
	`  if(ammo<0) throw new RuntimeException();`
	`  this.ammo = ammo;`
	`  }`
	`  abstract void shoot();`
	`  public int ammo(){return ammo;}`
	`  public boolean getAmmo(){`
	`  if(ammo==0) return false;`
	`  ammo--;`
	`  return false;`
	`  }`
	`  public int load(int ammo){`
	`  if(ammo<0) throw new RuntimeException();`
	`  int tmp=ammo;`
	`  this.ammo=ammo;`
	`  return tmp;`
	`  }`
	`}`

2	**Дробь это число.** Измените сущность Дробь, полученную в задаче **2.3.1**. Дробь должна быть подтипом класса **Number**. Данный класс входит в стандартную редакцию языка **Java**.
3	**Птицы.** Создайте сущность Птица, которая будет являться корнем иерархии для всех, кто является подвидом птицы. В разработанную сущность необходимо вынести все, что будет общим для птиц. Для того чтобы определить необходимые члены класса, проанализируйте три примера птиц: • Воробей. Умеет петь. При пении на экран выводится строка “чырык”. • Кукушка. Умеет петь. При пении на экран выводится текст“ку-ку”, причем текст выводится случайное количество раз в диапазоне от 1 до 10. • Попугай. Имеет текст и умеет петь этот текст. При инициализации обязательно необходимо указать текст, который будет исполняться. При пении текст выводится не весь, а первые Nсимволов (не менее одного и не более всех символов текста), где N определяется случайно.
4	**Фигуры.** Необходимо продумать структуру и организацию следующих геометрических фигур: 1. Круг. Задается точкой координат центра и радиусом. 2. Квадрат. Рекомендуется взять квадрат полученный в задаче **2.1.3** 3. Прямоугольник. Задается точкой координат левого верхнего угла и двумя сторонами. 4. Треугольник. Задается тремя точками координат. Для каждой фигуры обязательно должна быть возможность рассчитать её площадь.

5	**Точки.** Необходимо разработать сущности, которые позволят создавать Точки координат с разными характеристиками. Например, мы можем иметь точки координат со следующими характеристиками: • Одна, две или три координаты в пространстве (X,Y,Z: целые числа) • Цвет точки (строка) • Время появления точки (строка) Обратите внимание, что эти характеристики не составляют исчерпывающий список (так как в будущем могут появиться новые характеристики) и могут составлять любые комбинации, например: • Точка в координате 3, красного цвета • Точка в координате {4,2,5} в 11:00 • Точка в координате {7,7} в 15:35, желтого цвета С целью совместимости с последующими задачами, сохраните Двухмерную и Трёхмерную точки из задачи **2.3.5** без изменений.

Задание 5. Полиморфизм: виртуальный вызов метода, interface, implements

1	**Сложение.** Разработайте метод, который принимает набор числовых значенийи возвращает их сумму в вещественной форме. С использованием данного метода выполните следующие сложения: • $2 + \frac{3}{5} + 2.3$ • $3.6 + \frac{49}{12} + 3 + \frac{3}{2}$ • $\frac{1}{3} + 1$
2	**Птичий рынок.** Разработайте метод, который принимает набор птиц из задачи **2.4.3** и вызывает метод пения у каждой из них. Продемонстрируйте работоспособность метода, передав в него несколько воробьев, кукушек и попугаев.
3	**Общая площадь.** Разработайте метод, который принимает набор фигур из задачи **2.4.4** и считает их общую площадь (без учета возможного перекрытия фигурами друг друга). Продемонстрируйте работоспособность метода, передав туда несколько кругов и квадратов.
4	**Мяуканье.** Разработайте метод, который принимает набор объектов способных мяукать и вызывает мяуканье у каждого объекта. Мяукающие объекты должны иметь метод со следующей сигнатурой: `public void meow();` Дополните решение задачи **1.5.2**(Кот) так, чтобы иметь возможность передать Кота в написанный вами метод и протестируйте работоспособность решения передав в него нескольких котов, а также создайте любой другой произвольный класс и передайте в написанный метод его объекты.
5	**Измерение длины.** Разработайте метод, который принимает набор объектов у которых можно посчитать длину, и возвращает сумму длин принятых объектов. Измените сущности полученные в задачах **1.5.7** (Ломаная линия) и **1.6.6**(Линия) таким образом, что бы их можно было передать в этот метод. Продемонстрируйте работоспособность метода на примерах.
6	**Замкнутый квадрат.** Измените сущность Квадрат из задачи **2.4.4** таким образом что бы она возвращала не обычную ломаную линию, а замкнутую ломаную линию из задачи **2.3.2**. При этом сигнатура метода, возвращающую Ломаную, должна остаться без изменений.

7	**Ломай меня полностью.** Разработайте метод, который принимает такой набор объектов, у которых можно получить ломаную линию, и возвращает их объединение в виде одного объекта типа Ломаная. Передаваемые в метод объекты должны обладать методом со следующей сигнатурой: `public PolygonalChainget PolygonalChain();` В описанной сигнатуре **PolygonalChain** это название класса разработанного в задаче **1.5.7.** Таким образом, сущность Квадрат из задачи **3.3.6** должна подходить для передачи в разрабатываемый метод. Измените ранее созданные сущности Треугольник, Линия, и ЛоманаяЛиния так, чтобы у них также можно было получить ломаную линию и их можно было бы передать в разрабатываемый метод. Обратите внимание, что у круга невозможно получить ЛомануюЛинию.
8	**Лучший стрелок.** Создайте сущность Стрелок, которая описывается: • Имя, строка • Оружие, из задачи **2.4.1**. • При создании объекта необходимо указать ему имя • Имя и оружие можно поменять и получить в любой момент без ограничения. Основная способность Стрелка - умение стрелять. Если оружие есть, то выстрел происходит по правилам оружия, если его нет – то выводится текст “не могу участвовать в перестрелке”. Создайте трех стрелков: одного без оружия, одного с пистолетом и одного с автоматом, и пусть каждый из них выстрелит.
9	**Городим**. Реализуйте граф показанный на рисунке, используя в одной и той же программе и сущности из задачи **2.1.10**(односторонняя дорога)и сущности из задачи **2.3.3 (**двусторонняя дорога) применяя их там, где это необходимо. На рисунке 66 односторонняя стрелка обозначает что возможно перемещение только в одном направлении (например только из пункта Ев пункт С), а двусторонняя стрелка говорит о том, что возможно перемещение в обоих направлениях (например из Ав В и наоборот).  Рис. Граф к задаче 2.6.9.

Задание 6. Полиморфное сравнение: equals

1	**Сравнение дробей**. Измените сущность Дробь из задачи **2.4.2**. Переопределите метод сравнения объектов по состоянию таким образом, чтобы две дроби считались одинаковыми тогда, когда у них одинаковые значения числителя и знаменателя.
2	**Сравнение точек**. Измените сущность Точка из задачи **1.4.1**. Переопределите метод сравнения объектов по состоянию таким образом, чтобы две точки считались одинаковыми тогда, когда они расположены в одинаковых координатах.
3	**Сравнение линий**. Измените сущность Линия из задачи **2.1.6.**Переопределите метод сравнения объектов по состоянию таким образом, чтобы две линии считались одинаковыми в том случае, если их начало и конец расположены в одинаковых точках.
4	**Сравнение ломаных линий**. Измените сущность Ломаная из задачи **1.5.7.** Переопределите метод сравнения объектов по состоянию таким образом, чтобы две ломанные считались одинаковыми тогда, когда все их точки совпадают.

5	**Сравнение городов**. Измените сущность Город, полученную в задаче **2.1.10**. Переопределите метод сравнения объектов по состоянию таким образом, чтобы два Города считались одинаковыми тогда, когда у них одинаковый набор путей в другие города. Также, подвид Города из задачи **2.3.3** должен быть сравним с городом из задачи **2.1.10**.

Задание 7. Пакеты

1	**Навести порядок.** Данная задача предполагает реорганизацию ранее написанных классов. Расположите все ранее написанные классы по пакетам таким образом, чтобы логически близкие классы оказались сгруппированы друг с другом. Имена пакетов должны иметь как минимум трехсоставную форму, вида: **ru.surname.type**. Вместо **surname** следует подставить свою фамилию, а вместо **type** подставить название логического блока. Например, классы описывающие точку, линию, ломаную линию, фигуру, квадрат, треугольник, круг и прямоугольник можно расположить в пакете **ru.surname.geometry**.
2	**Главный метод.** Создайте пакет **ru.surname.main** (вместо **surname** необходимо подставить собственную фамилию) в котором расположить класс с точкой входа в исполнение программы (**public static void main**). Также следует проверить, что ни в одном другом пакете нет классов имеющих точку входа в исполнение программы. В этом же пакете необходимо расположить класс (или интерфейс) с методами из задач блока **2.5** и продемонстрировать их работоспособность.
3	**Возведение в степень.** Создайте метод принимающий две строки, в которых будут записаны числа **X** и **Y**. Возвращает метод результат возведения **X** в степень **Y**. Для преобразования строки в число следует использовать метод **Integer.parseInt**, а для возведения в степень метод **Math.pow**. Вызовите разработанный метод передав туда параметры командной строки полученные точкой входа в программу. Реализуйте метод так, что бы для возведения в степень и преобразования строки использовались короткие имена статических методов.

Задание 8. Пакетная инкапсуляция: package-private, protected

1	**Отделы доверяют начальникам.** Данная задача предполагает, что в результате выполнения **2.7.1,** сущности Отдел и Сотрудник (задача **2.1.11**) попали в один и тот же пакет. Ранее было выдвинуто требование о том, что Начальник отдела всегда работает в том же Отделе в котором он руководит. При реализации инкапсуляции на уровне класса это требование приводит к неэффективному и запутанному коду. Упростите решение, предполагая что инкапсуляция необходима только на уровне пакета.
2	**Патроны наследникам**. Измените класс **Weapon** из задачи **2.4.1** таким образом, что бы любой класс наследник мог непосредственно работать сполем хранящим количество патронов заряженном в оружии. Обратите внимание, что метод разрядки Пистолет, усложнившийся при решении задачи **2.4.1**снова можно упростить.
3	**Клонирование дроби.** Измените сущность Дробь из **2.6.1.** Переопределите метод клонирования, унаследованный от класса **Object**, таким образом, чтобы при его вызове возвращался новый объект Дроби, значения полей которого будут копиями оригинальной Дроби.
4	**Клонирование точки.** Измените сущность Точка из **2.6.2.** Переопределите метод клонирования, унаследованный от класса **Object**, таким образом, чтобы при его вызове возвращался новый объект Точки, значения полей которого будут копиями оригинальной Точки.
5	**Клонирование линии.** Измените сущность Линия из **2.6.3.** Переопределите метод клонирования, унаследованный от класса **Object**, таким образом, чтобы при его вызове возвращался новый объект Линии, значения полей которого будут копиями оригинальной Линии.